



Hochschule
Zittau/Görlitz
UNIVERSITY OF APPLIED SCIENCES

Wie das Web funktioniert

Web Engineering 1 (319300)

Vorlesung 01

M.Sc. Christopher-Manuel Hilgner

Hochschule Zittau/Görlitz

christopher.hilgner@stud.hszg.de

2026-10-06

- Vorlesungen + Seminar
- Projektarbeit im Laufe des Semesters
- Crossbeleg mit anderen Modulen wird empfohlen
- Beleg am Ende des Semesters abgeben
- Zur Abgabe gehören:
 - Beleg als PDF
 - Alle Projektdateien in einem Git-Repository
 - Video, das die Funktionalität des Projektes zeigt

Beleg

- Keine festgelegte Seitenanzahl (10-15 Seiten als Orientierung)
- Anforderungsanalyse
- Beschreibung der Umsetzung
- Gründe für Entscheidungen bei der Entwicklung darstellen
- Dokumentation der Software

Material

<https://github.com/XPromus/we1>

- Folien, Seminare, Beispielprojekte ...

Inhalte

- Grundlagen des Webs
- Frontend Entwicklung
- Backend Entwicklung

Agenda

- Client-Server-Modell
- Request-Response Zyklus
- Bestandteile einer URL
- Ablauf vom Aufruf einer URL bis zur gerenderten Seite
- HTTP Request und Response
- HTTP Methoden
- Status Codes

Client-Server-Modell

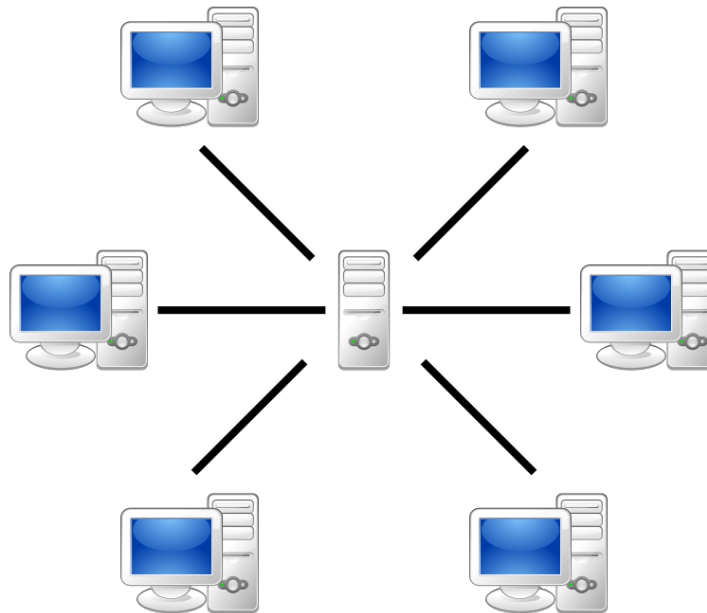


Abbildung 1: Server-basiertes Netzwerk

- Verteilung von Aufgaben innerhalb eines Netzwerks
- Unterteilung der Computer in Clients und Server
- Client fordert Services vom Server an
- Server beantwortet Request

Client vs Server

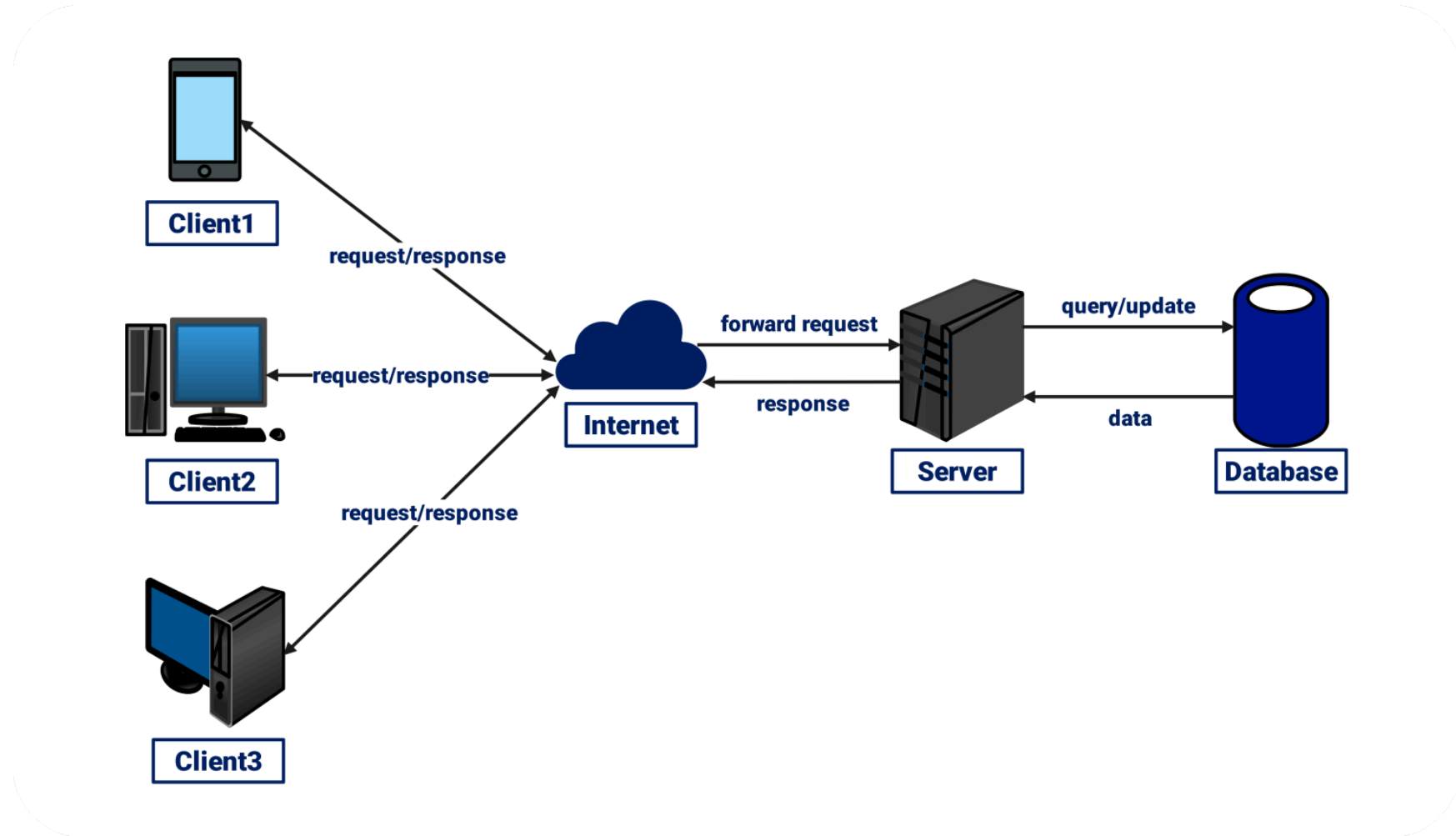


Abbildung 2: Unterschiedliche Clients verbinden sich über das Internet mit einem Server

Server

- Ein Programm, das mit dem Client kommuniziert
- Server bietet Zugang zu einem Dienst für den Client
- Server ist eine Rolle
- Definierbar auf
 - Geräteebene (Server-Host)
 - Softwareebene (Server-Applikation)
- Host kann sowohl Client als auch Server sein

Client

- Client fordert Service vom Server an
- Programm, das eine Request initialisiert
- Im Web-Kontext: Ein Browser
- Auch Apps, andere Server oder Tools wie curl

Request-Response-Zyklus

Jede Interaktion folgt der gleichen Abfolge

- Der Client schickt eine Request
- Der Server schickt genau eine Response
- Wenn der Client neue Daten braucht, muss eine neue Request geschickt werden

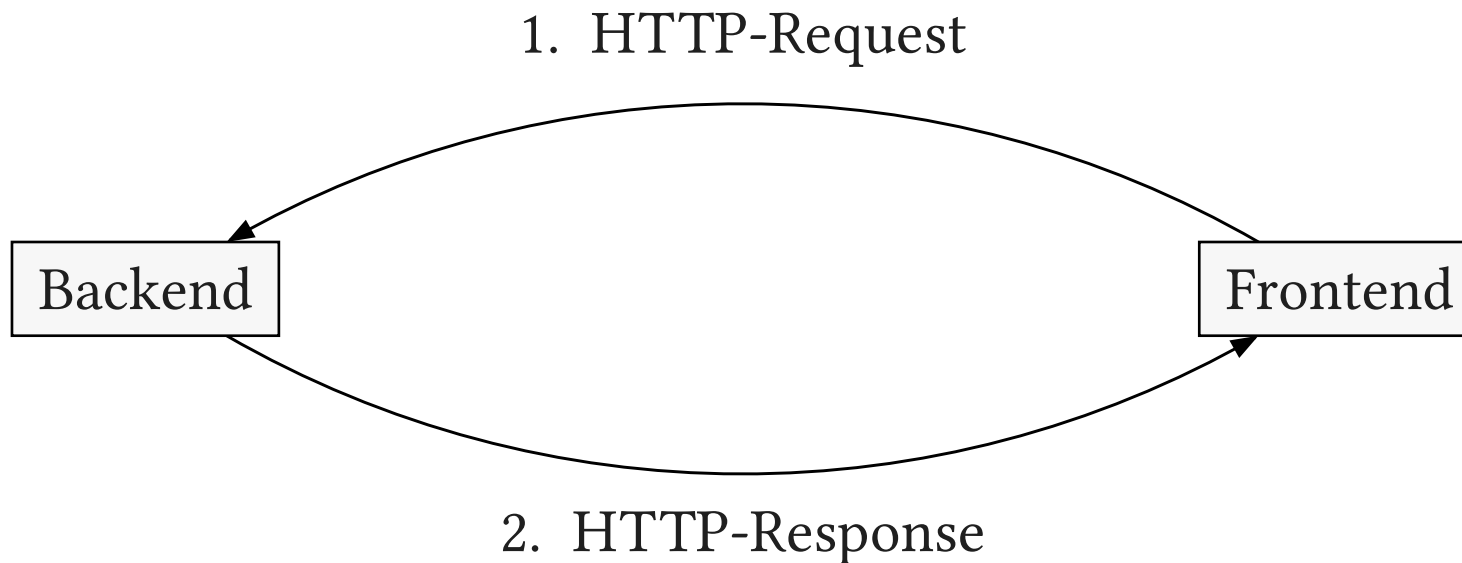


Abbildung 3: Der Request-Response-Zyklus

HTTP Statelessness

- Jede Request wird vom Server behandelt, als hätte der Server keine Erinnerungen an alte Requests
- Zwei Requests vom gleichen Browser sind für HTTP komplett unzusammenhängend
- **Ziel:** Einfacher Aufbau einer Request und Skalierbarkeit

Wie kann eine Website sich merken, dass ein User eingeloggt ist?

Nutzung von Cookies, Sessions und JSON Web Tokens, um Kontext zu halten.

Von der URL zur Seite

Aufbau einer URL

1 `https://api.example.com:443/users/5?active=true#profile`

- `https://`: Genutztes Protokoll

Aufbau einer URL

1 `https://api.example.com:443/users/5?active=true#profile`

- `https://`: Genutztes Protokoll
- `api.example.com`: Adresse des Hosts (Welcher Server wird angesprochen)

Aufbau einer URL

1 `https://api.example.com:443/users/5?active=true#profile`

- `https://`: Genutztes Protokoll
- `api.example.com`: Adresse des Hosts (Welcher Server wird angesprochen)
- `:443`: Port der Applikation auf dem Server

Aufbau einer URL

1 `https://api.example.com:443/users/5?active=true#profile`

- `https://`: Genutztes Protokoll
- `api.example.com`: Adresse des Hosts (Welcher Server wird angesprochen)
- `:443`: Port der Applikation auf dem Server
- `/users/5`: Pfad (Welche Ressource auf dem Server)

Aufbau einer URL

1 `https://api.example.com:443/users/5?active=true#profile`

- `https://`: Genutztes Protokoll
- `api.example.com`: Adresse des Hosts (Welcher Server wird angesprochen)
- `:443`: Port der Applikation auf dem Server
- `/users/5`: Pfad (Welche Ressource auf dem Server)
- `?active=true`: Query String mit Parametern

Aufbau einer URL

1 `https://api.example.com:443/users/5?active=true#profile`

- `https://`: Genutztes Protokoll
- `api.example.com`: Adresse des Hosts (Welcher Server wird angesprochen)
- `:443`: Port der Applikation auf dem Server
- `/users/5`: Pfad (Welche Ressource auf dem Server)
- `?active=true`: Query String mit Parametern
- `#profile`: Fragment welches beim Client verbleibt

Beispiele

1 `https://github.com/typst/typst#example`

Benenne die URL Komponenten

Beispiele

1 `https://github.com/typst/typst#example`

Benenne die URL Komponenten

<code>https://</code>	Genutztes Protokoll
<code>github.com</code>	Adresse des Hosts
<code>/typst/typst</code>	Pfad zur Ressource
<code>#example</code>	Fragment welches beim Client verbleibt

Beispiele

Beispiele

```
1 https://github.com/search?q=Typst&type=repositories
```

Benenne die URL Komponenten

Beispiele

Beispiele

1 `https://github.com/search?q=Typst&type=repositories`

Benenne die URL Komponenten

<code>https://</code>	Genutztes Protokoll
<code>github.com</code>	Adresse des Hosts
<code>/search</code>	Pfad zur Ressource
<code>?q=Typst&type=repositories</code>	Query String mit Parametern (hier für die Suche)

DNS

- *Das Telefonbuch des Internets*
- Computer kennen keine Domains, nur IP-Adressen
- Bei Angabe einer Domain wird ein DNS-Resolver gefragt, welche IP-Adresse dahinter liegt

DNS

- *Das Telefonbuch des Internets*
- Computer kennen keine Domains, nur IP-Adressen
- Bei Angabe einer Domain wird ein DNS-Resolver gefragt, welche IP-Adresse dahinter liegt

TCP Handshake

- Vor HTTP-Versendung muss stabile Verbindung hergestellt werden
- Drei Schritte:
 1. Bist du da?
 2. Ja ich bin da!
 3. Lass uns reden!

- HTTP allein schickt nur in Klartext → jeder kann Daten lesen
- HTTPS *umschließt* HTTP in einem verschlüsselter Tunnel (TLS)
- Der Inhalt ist während der Übertragung unlesbar



Abbildung 4: Markierung im Browser, dass eine Website HTTPS benutzt. Es sagt jedoch nicht aus, dass die Seite selbst sicher ist.

1. User öffnet URL
2. DNS wandelt den Host zu einer IP um
3. TCP Handshake etabliert die Verbindung
4. TLS Handshake verschlüsselt die Verbindung, sofern HTTPS genutzt wird
5. Browser schickt HTTP Request
6. Server verarbeitet die Request
7. Browser wandelt Response um und rendert Website

Beispiel

Ein User berichtet, dass eine Website nicht lädt. Die DevTools zeigen, dass die Request nie den Browser verlassen hat.

Bei welchen Schritten des Ablaufs kann das Problem liegen?

Ursache	Typische Fehlermeldung
DNS (Schritt 2): Domain lässt sich nicht auflösen	ERR_NAME_NOT_RESOLVED
TCP (Schritt 3): Server nicht erreichbar, Port gesperrt, Firewall	ERR_CONNECTION_REFUSED, ERR_CONNECTION_TIMED_OUT
TLS (Schritt 4): Zertifikat abgelaufen oder ungültig	ERR_CERT_DATE_INVALID
Client: kein Netzwerk, Adblocker oder Extension	ERR_INTERNET_DISCONNECTED, ERR_BLOCKED_BY_CLIENT

Merke: Auch ein ausgefallener Server kann die Ursache sein, nämlich wenn der Fehler schon beim TCP Handshake auftritt.

HTTP Request

Request-Response-Zyklus

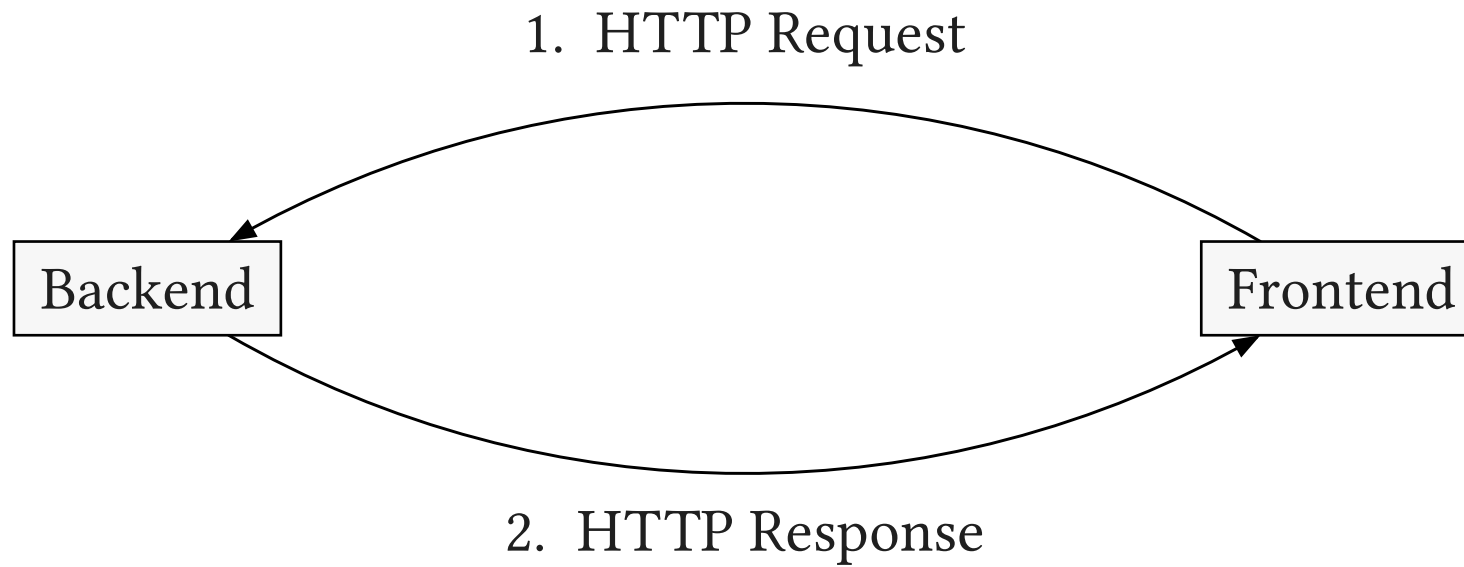


Abbildung 5: Der Request-Response-Zyklus

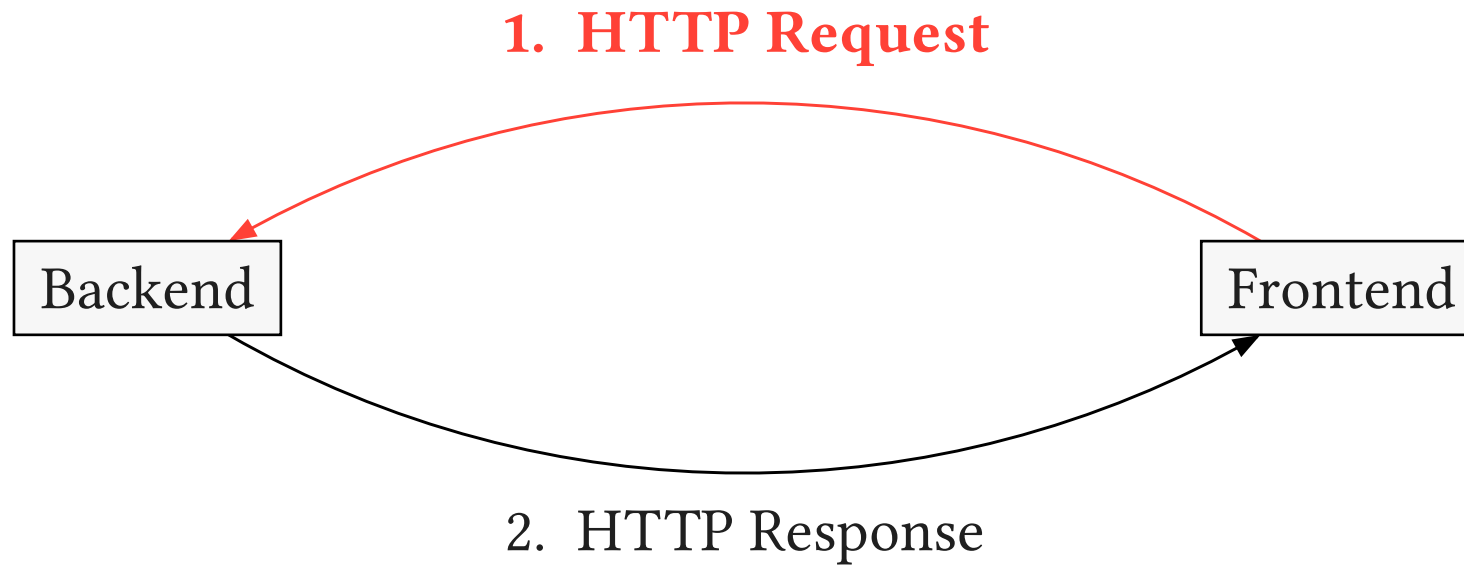


Abbildung 6: Der Request-Response-Zyklus

- **Request Line** mit folgenden Bestandteilen:
 - Ein **HTTP Verb**, das die Art der Operation definiert
 - Einen **Pfad** zu einer Ressource
 - **HTTP-Version**
- Einen **Header**, der Informationen über die Request enthält
- Einen optionalen **Body**, der weitere Daten enthält

Beispiel

```
1  POST http://localhost:8080/todos HTTP/1.1
2  Accept: */*
3  Content-Type: application/json
4
5  {
6      "name": "todoItemName",
7      "description": "newDescription",
8      "done": true,
9      "created": "2025-10-23T15:06:08.738Z"
10 }
```

Beispiel

1	<code>POST http://localhost:8080/todos HTTP/1.1</code>	} (1)	Request Line						
2	<code>Accept: */*</code>	}		(2)					
3	<code>Content-Type: application/json</code>		}		(2)				
4									
5	<code>{</code>	}	(3)						
6	<code> "name": "todoItemName",</code>			}	(3)				
7	<code> "description": "newDescription",</code>					}	(3)		
8	<code> "done": true,</code>							}	(3)
9	<code> "created": "2025-10-23T15:06:08.738Z"</code>								
10	<code>}</code>	}	(3)						

Request Line

```
1 POST http://localhost:8080/todos HTTP/1.1
```

- POST: Gibt die genutzte HTTP-Methode an
- `http://localhost:8080/todos`: Request-Target gibt an, an welche Ressource die Anfrage gerichtet ist
- HTTP/1.1: HTTP Version gibt an, welche Version des HTTP-Protokolls genutzt wird

Hinweis: Bei HTTP/1.1 ist normalerweise nur der Pfad enthalten:

```
1 POST /todos HTTP/1.1
```

```
2 Host: localhost:8080
```

Volle URL wird meist von Tools wie curl, Postman oder Browser-Entwicklertools angezeigt.

- Definiert, auf welcher Ressource die Operation ausgeführt werden soll
- Erster Teil sollte die Pluralform der Ressource sein

Beispiel: `store.com/customers/223/orders/12`

Lesbarkeit der Pfade

Richtlinie 1

Jeder API Pfad sollte auch ohne Kenntnisse über das System dahinter klar machen, was bei der Request passieren wird.

```
1  Accept: */*
2  Content-Type: application/json
```

- Accept: Welche Formate der Client akzeptiert
- Content-Type: Welches Format besitzen die Daten, die der Client schickt
- Art der Ressource über MIME Types

MIME Type:

```
1  type/subtype;parameter=value
```

- parameter ist optional
- **Beispiele:** image/png, audio/wav, application/json

HTTP Response

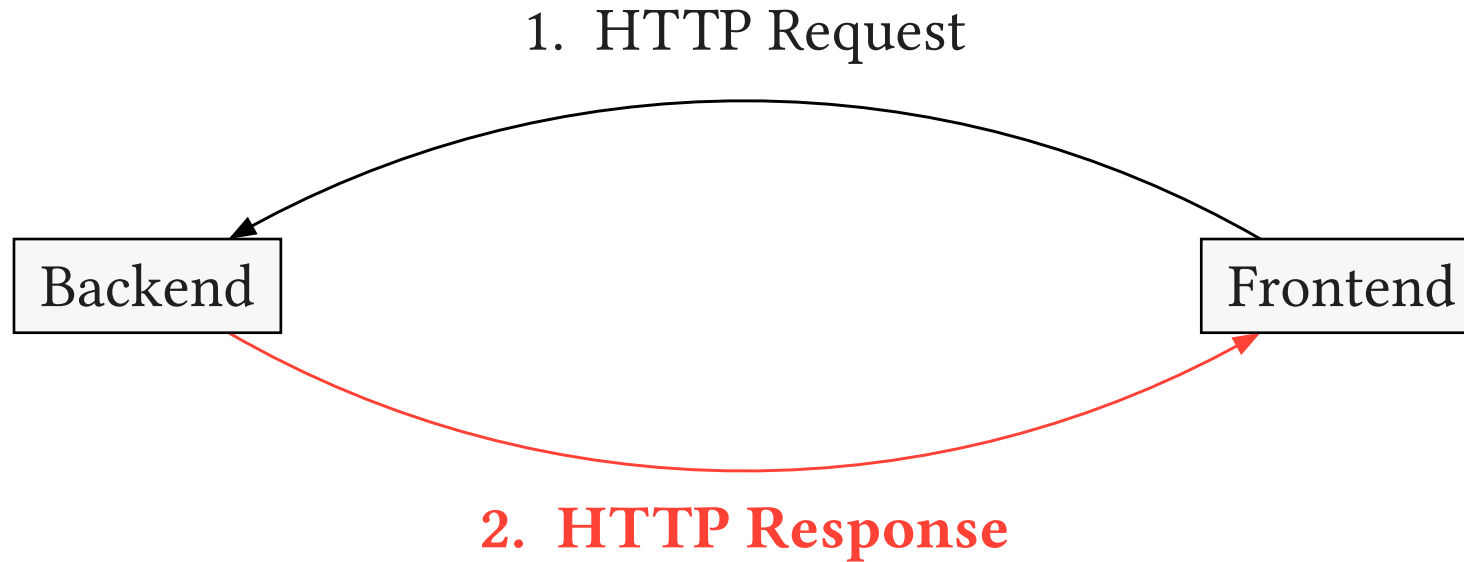


Abbildung 7: Der Request-Response-Zyklus

- Datentyp angeben wenn Daten zurückgegeben werden sollen
- Content-Type im Header wie bei Request
- Status Code anhängen für Informationen über Ausgang der Request

Beispiel

```
1  HTTP/1.1 200 OK
2  Connection: keep-alive
3  Content-Type: application/json
4  Date: Thu, 23 Oct 2025 16:45:09 GMT
5  Keep-Alive: timeout=60
6  Transfer-Encoding: chunked
7
8  [
9    {
10      "id": 0,
11      // Mehr Daten
12    }
13  ]
```

Beispiel

```
1 HTTP/1.1 200 OK } (1) Code
2 Connection: keep-alive
3 Content-Type: application/json } (2) Header
4 Date: Thu, 23 Oct 2025 16:45:09 GMT
5 Keep-Alive: timeout=60
6 Transfer-Encoding: chunked
7
8 [ } (3) JSON Body
9 {
10     "id": 0,
11     // Mehr Daten
12 }
13 ]
```

HTTP Methoden

Definition von Verben, die Ziele von Requests genauer beschreiben

Häufige Verben

- GET
- POST
- PUT
- DELETE

Weitere Verben

- HEAD
- CONNECT
- OPTIONS
- TRACE
- PATCH

- Stellt einen Request an den Server, um eine Ressource zu transferieren
- Ist safe und idempotent
- Content sollte nie mit einer GET-Request erstellt werden
- Caching ist möglich

Informationen in der URL: Es ist zu beachten, dass, wenn Ressourcen nur über URLs angefragt werden, potentiell sicherheitskritische Informationen in dieser URL landen können. Wenn es nicht möglich ist, diese Informationen in weniger kritische zu transformieren, wird das Nutzen einer POST Request mit den Daten im Request Content empfohlen.

- Wird genutzt, um transferierte Daten nach Server-Spezifikation zu verarbeiten
- Beispiel:
 - Daten in Inputfeldern übergeben
 - Nachrichten posten (Foren, Social Media usw.)
 - Erstellen von neuen Ressourcen
 - Daten an vorhandene Ressourcen anhängen
- Server kommuniziert mit Status Codes das Ergebnis der POST Request
- Bei Erfolg: 200 oder 204
- Bei erfolgreichem Erstellen einer neuen Ressource: 201 mit Pfad zur neuen Ressource

- Editieren von vorhandener Ressource oder Erstellung von neuen Ressourcen
- Request basiert auf mitgeschickten Daten
- Wenn Ressource nicht vorhanden ist, wird sie neu erstellt
- Status Code 201 nach Erstellen neuer Ressource
- Wenn kein neuer Eintrag erstellt wurde: Status Code 200 oder 204
- Server sollte Daten in der PUT Request validieren
- Wenn Fehler in den Daten: selbst versuchen Daten in passendes Format zu bringen oder 400, 409 oder 422 zurückgeben

- Ändert eine vorhandene Ressource teilweise
- Body enthält nur die Änderungen
- Im Gegensatz dazu ersetzt PUT die Ressource vollständig, fehlende Felder gehen verloren
- Nicht safe, und nicht garantiert idempotent (hängt von der Änderung ab)
- **Erfolg**: 200 (mit aktualisierter Ressource) oder 204
- **Fehler**: 404 (Ressource existiert nicht), 409 oder 422 (Änderung nicht anwendbar oder ungültig)

- Request an den Server, Ressource zu löschen
- Entweder Entfernen von Referenzen oder komplettes Löschen der Ressource
- DELETE sollte nur bei Ressourcen erlaubt sein, die definierte Löschoperationen besitzen
- Bei Erfolg einer der folgenden Codes:
 - **202 (Accepted)** wenn das Löschen wahrscheinlich erfolgreich sein wird, aber noch nicht durchgeführt wurde
 - **204 (No Content)** Löschen wurde ausgeführt und keine weiteren Informationen sind nötig
 - **200 (OK)** Löschen war erfolgreich und die Response enthält noch Informationen über den aktuellen Status

- Eine Methode ist *safe*, wenn sie nur lesen soll und keine Zustandsänderungen auf dem Server beabsichtigt
- Der Client fordert keine Änderung an und ist nicht dafür verantwortlich
- Safe: GET, HEAD, OPTIONS, TRACE
- Server darf Side Effects haben, solange sie nicht Änderungen an Ressourcen vornehmen auf die sich der Client verlässt (z.B.: Logs, Counter, Cache)

- Eine Methode ist *safe*, wenn sie nur lesen soll und keine Zustandsänderungen auf dem Server beabsichtigt
- Der Client fordert keine Änderung an und ist nicht dafür verantwortlich
- Safe: GET, HEAD, OPTIONS, TRACE
- Server darf Side Effects haben, solange sie nicht Änderungen an Ressourcen vornehmen auf die sich der Client verlässt (z.B.: Logs, Counter, Cache)

Folgen: Safe Methods können gefahrlos gecached werden, vorab geladen und wiederholt werden.

Safe Methods

- Eine Methode ist *safe*, wenn sie nur lesen soll und keine Zustandsänderungen auf dem Server beabsichtigt
- Der Client fordert keine Änderung an und ist nicht dafür verantwortlich
- Safe: GET, HEAD, OPTIONS, TRACE
- Server darf Side Effects haben, solange sie nicht Änderungen an Ressourcen vornehmen auf die sich der Client verlässt (z.B.: Logs, Counter, Cache)

Folgen: Safe Methods können gefahrlos gecached werden, vorab geladen und wiederholt werden.

Hinweis: *Safe* ist ein Versprechen, keine Garantie. GET Endpoints können theoretisch so geschrieben werden, dass sie Daten löschen.

Idempotente Methoden

- Mehrfache Ausführung hat den gleichen Effekt auf dem Server
- Wichtig bei automatischen Requests (z.B.: Wiederholung bei Fehlschlag)
- PUT und DELETE sollen nach Spezifikation idempotent sein
- POST und PATCH sind nicht idempotent
- *safe request methods* sind idempotent
- Server kann trotzdem Side Effects einfügen (z.B.: Logs)
- Side Effects dürfen Ergebnis nicht verändern
- Nicht idempotente Methoden sollten nicht automatisch wiederholt werden (Außer, wenn die Implementierung idempotent ist)

Idempotent vs. Safe

- *Idempotent*: Mehrfache Ausführung hat den gleichen Effekt auf dem Server
- *Safe*: Methode verändert den Server-Zustand gar nicht (aus Sicht des Clients)
- Jede safe Methode ist idempotent - nicht jede idempotente Methode ist safe

Methode	Safe	Idempotent
GET	ja	ja
HEAD	ja	ja
OPTIONS	ja	ja
PUT	nein	ja
DELETE	nein	ja
POST	nein	nein
PATCH	nein	nein (nicht garantiert)

```
1 DELETE /todos/5
```

- Nicht safe, denn es löscht Einträge vom Server
- Ist idempotent, denn nach dem ersten Ausführen ist der Eintrag weg und der Zustand ändert sich nicht

```
1 POST /todos
```

- Weder safe noch idempotent
- Jeder Aufruf erzeugt neuen Eintrag

HTTP Status Codes

HTTP Status Codes

- Status Code gehört zu jeder HTTP Response
- Zwischen 100 und 599
- Erste Ziffer gibt Klasse der Response an
 - **1xx (Informational)**: Die Request wurde erhalten und wird verarbeitet

HTTP Status Codes

- Status Code gehört zu jeder HTTP Response
- Zwischen 100 und 599
- Erste Ziffer gibt Klasse der Response an
 - **1xx (Informational)**: Die Request wurde erhalten und wird verarbeitet
 - **2xx (Successful)**: Die Request wurde erfolgreich erhalten, verstanden und akzeptiert

HTTP Status Codes

- Status Code gehört zu jeder HTTP Response
- Zwischen 100 und 599
- Erste Ziffer gibt Klasse der Response an
 - **1xx (Informational)**: Die Request wurde erhalten und wird verarbeitet
 - **2xx (Successful)**: Die Request wurde erfolgreich erhalten, verstanden und akzeptiert
 - **3xx (Redirection)**: Es müssen weitere Schritte durchgeführt werden, damit die Request verarbeitet werden kann

HTTP Status Codes

- Status Code gehört zu jeder HTTP Response
- Zwischen 100 und 599
- Erste Ziffer gibt Klasse der Response an
 - ▶ **1xx (Informational)**: Die Request wurde erhalten und wird verarbeitet
 - ▶ **2xx (Successful)**: Die Request wurde erfolgreich erhalten, verstanden und akzeptiert
 - ▶ **3xx (Redirection)**: Es müssen weitere Schritte durchgeführt werden, damit die Request verarbeitet werden kann
 - ▶ **4xx (Client Error)**: Die Request enthält falsche Syntax oder kann nicht erfüllt werden

HTTP Status Codes

- Status Code gehört zu jeder HTTP Response
- Zwischen 100 und 599
- Erste Ziffer gibt Klasse der Response an
 - ▶ **1xx (Informational)**: Die Request wurde erhalten und wird verarbeitet
 - ▶ **2xx (Successful)**: Die Request wurde erfolgreich erhalten, verstanden und akzeptiert
 - ▶ **3xx (Redirection)**: Es müssen weitere Schritte durchgeführt werden, damit die Request verarbeitet werden kann
 - ▶ **4xx (Client Error)**: Die Request enthält falsche Syntax oder kann nicht erfüllt werden
 - ▶ **5xx (Server Error)**: Der Server konnte eine eigentlich valide Request nicht erfüllen

HTTP Status Codes

Status Code	Bedeutung
200 (OK)	Standardantwort für Erfolg
201 (CREATED)	Standardantwort für neue Ressource
204 (NO CONTENT)	Standardantwort für Erfolg ohne Daten im Response Body
301 (MOVED PERMANENTLY)	Ressource ist an der Adresse im „Location“-Header-Feld
302 (FOUND) (MOVED TEMPORARILY)	Ressource ist vorübergehend an der Adresse im „Location“-Header-Feld
400 (BAD REQUEST)	Die Request konnte nicht verarbeitet werden
403 (FORBIDDEN)	Der Client hat keine Rechte auf diese Ressource
404 (NOT FOUND)	Die gewünschte Ressource konnte nicht gefunden werden
409 (CONFLICT)	Die Anfrage steht im Konflikt mit dem aktuellen Zustand der Ressource (z.B. gleichzeitige Änderung durch Dritte)
415 (UNSUPPORTED MEDIA TYPE)	Inhalt der Anfrage wurde mit ungültigem Medientyp übermittelt.
422 (UNPROCESSABLE CONTENT)	Die Anfrage ist formal korrekt, der Inhalt aber fachlich ungültig
500 (INTERNAL SERVER ERROR)	Standardantwort für einen unerwarteten Fehler

Zusammenfassung

- Ein **Client** fordert Dienste an, ein **Server** beantwortet die Anfrage
- Host kann Client und Server sein
- Jede Interaktion folgt dem Muster **Request, dann Response**
- Neue Daten brauchen neue Request
- HTTP ist **zustandslos**: Kontext entsteht erst durch Cookies, Sessions oder JSON Web Tokens
- Eine URL besteht aus: Protokoll, Host, Port, Pfad, Query String und Fragment
- Ablauf beim Aufruf: DNS, TCP Handshake, TLS Handshake, HTTP Request, Verarbeitung, Rendering
- HTTPS verschlüsselt Übertragung

- Request besteht aus: Request Line (Verb, Pfad und Version), Headern und optionalem Body
- Response besteht aus: Status Line, Headern und optionalem Body (Content-Type beschreibt den Datentyp)
- **GET** liest, **POST** erstellt und verarbeitet, **PUT** ersetzt oder erstellt, **PATCH** ändert teilweise, **DELETE** löscht
- **Safe** heißt: keine beabsichtigte Zustandsänderung
- **Idempotent** heißt: mehrfaches Ausführen wirkt wie einmaliges
- Jede safe Methode ist idempotent, aber nicht jede idempotente Methode ist safe
- **Status Codes:** 2xx Erfolg, 3xx Umleitung, 4xx Fehler des Clients, 5xx Fehler des Servers

Übersicht: HTTP Methoden

Methode	Safe	Idempotent	Erfolg	Fehler
GET	ja	ja	200	404, 403
HEAD	ja	ja	200	404, 403
OPTIONS	ja	ja	200	403
POST	nein	nein	200, 201, 204	400, 409, 422
PUT	nein	ja	200, 201, 204	400, 409, 422
PATCH	nein	nein (nicht garantiert)	200, 204	404, 409, 422
DELETE	nein	ja	200, 202, 204	404, 403, 409